

JDBC Development with the Spring Framework

Thomas Risberg
Data Architecture, TargetRx, Inc.



Why?

Why do I use and advocate using Spring for JDBC development? -- Because:

- I want ease of development
- I want to be able to read my code without the distracting noise from try-catch-finally blocks
- I don't want to get calls when the connection pool runs out of connections
- I like to get meaningful exception I can handle in code without interpreting vendor specific codes

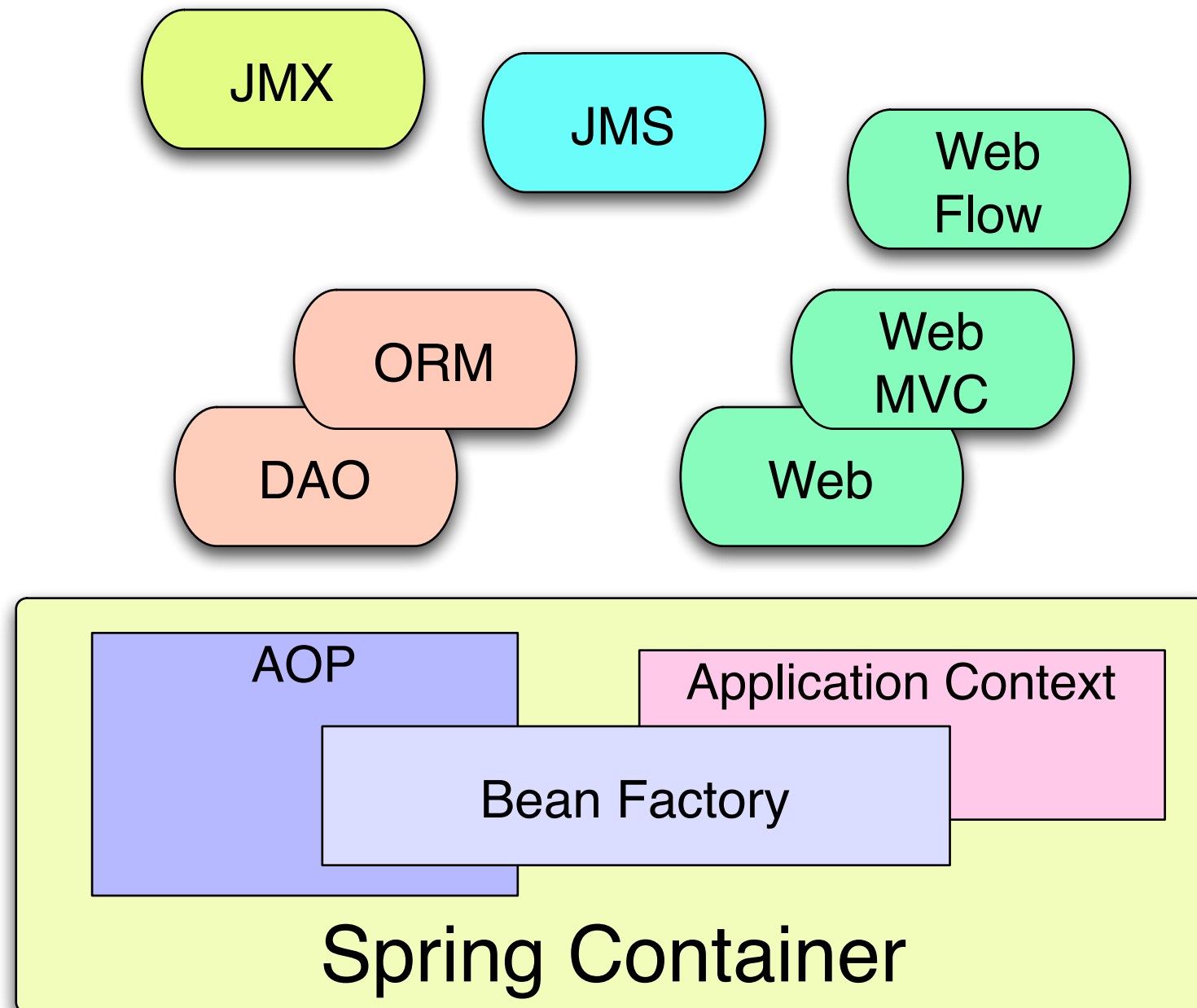
Outline

- Introduction - What is Spring? A short code example
- Comparing JDBC and Spring concepts
- Query and Update
- Stored Procedures
- Batch Processing
- Transaction Management
- Exception Handling
- Bootstrapping - standalone, servlet, utility, Spring MVC

Introduction

- ★ What is Spring? *Spring is an application framework built on a lightweight container architecture that helps you structure, configure, and layer your applications consistently.*

Introduction



Introduction

What do you need?

1. download Spring distribution from
www.springframework.org/download.html
2. unzip
spring-framework-1.1.5-with-dependencies.zip
3. copy the following to your lib directory:
spring-framework-1.1.5/dist/spring.jar
spring-framework-1.1.5/lib/jakarta-commons/
commons-logging.jar

First JDBC Example

```
import java.util.List;
import javax.sql.DataSource;
import org.springframework.jdbc.core.JdbcTemplate;

public class DataStuff {
    private DataSource dataSource;
    private JdbcTemplate jdbcTemplate;

    List getNames() {
        return jdbcTemplate.queryForList("select * from mytest");
    }

    public DataSource getDataSource() {
        return dataSource;
    }

    public void setDataSource(DataSource dataSource) {
        this.dataSource = dataSource;
        this.jdbcTemplate = new JdbcTemplate(dataSource);
    }
}
```

First JDBC Example

```
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;

public class JustStuff {

    public static void main(String[] args) {
        XmlBeanFactory ctx;

        ctx = new XmlBeanFactory(new ClassPathResource("dataAccessContext.xml"));
        DataStuff test = (DataStuff)ctx.getBean("dataStuff");

        System.out.println(test.getNames());

        System.out.println("OK!");

        ctx.destroySingletons();
    }
}
```


First JDBC Example

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN//EN"
    "http://www.springframework.org/dtd/spring-beans.dtd">
<beans>
  <bean id="dataStuff" class="DataStuff">
    <property name="dataSource"><ref local="dataSource"/></property>
  </bean>

  <bean id="dataSource"
    class="org.apache.commons.dbcp.BasicDataSource"
    destroy-method="close">
    <property name="driverClassName">
      <value>oracle.jdbc.driver.OracleDriver</value>
    </property>
    <property name="url">
      <value>jdbc:oracle:thin:@localhost:1521:mac10g</value>
    </property>
    <property name="username"><value>scott</value></property>
    <property name="password"><value>tiger</value></property>
  </bean>
</beans>
```

First JDBC Example

QueryForList returns:

- java.util.List *represents the ResultSet*
- java.util.Map *one Map per row*
 - key=JDBC Column name
 - value=what getObject() returns for this column

[{ID=1, NAME=Sven}, {ID=2, NAME=Maria}, {ID=1, NAME=John}]

(Control capitalization of column names via SQL statement)

JDBC vs. Spring

JDBC	Spring
DriverManager / DataSource	DataSource
Statement / PreparedStatement / CallableStatement	JdbcTemplate / SQLObjects
ResultSet / RowSet	List of POJOs or Maps / SQLRowSet

JDBC vs. Spring

<u>Task</u>	<u>Spring</u>	<u>You</u>
Connection Management	✓	
SQL		✓
Statement Management	✓	
ResultSet Management	✓	
Row Data Retrieval		✓
Parameter Declaration		✓
Parameter Setting	✓	
Transaction Management	✓	

JdbcTemplate - Query

- queryForInt, queryForLong, queryForObject, queryForList, queryForRowSet -- *predefined processing logic and return types*
- query -- *uses a callback interface where you can write your own processing logic*
 - **RowCallbackHandler**
 - **RowMapper**

JdbcTemplate - Update

- `update -- sql + [parameters]`
- `update -- PreparedStatementCreator/
PreparedStatementSetter`

JdbcTemplate Example

```
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.List;

import org.springframework.jdbc.core.RowMapper;
import org.springframework.jdbc.core.support.JdbcDaoSupport;

public class DataStuff extends JdbcDaoSupport {

    public List getNames() {
        String querySql = "select name, id from mytest where id >= ?";
        return getJdbcTemplate().query(querySql, new Object[] {new Long(2)}, new RowMapper() {

            public Object mapRow(ResultSet rs, int currentRowNum) throws SQLException {
                Person person = new Person();
                person.setId(rs.getLong("id"));
                person.setName(rs.getString("name"));
                return person;
            }

        });
    }

    public void updateAName(long id, String name) {
        String sql = "update mytest set name = ? where id = ?";
        getJdbcTemplate().update(sql, new Object[] {name, new Long(id)});
    }
}
```

JdbcTemplate Example

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class JustStuff {

    public static void main(String[] args) {
        ApplicationContext ctx;

        ctx = new ClassPathXmlApplicationContext("dataAccessContext.xml");
        DataStuff test = (DataStuff)ctx.getBean("dataStuff");

        System.out.println("Before: " + test.getNames());

        test.updateAName(3, "Eric");

        System.out.println("After: " + test.getNames());

        System.out.println("OK!");
    }
}
```

JdbcTemplate Example

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN//EN"
    "http://www.springframework.org/dtd/spring-beans.dtd">

<beans>
    <!-- Configurer that replaces ${...} placeholders with values from properties files -->
    <!-- (in this case the JDBC related properties) -->
    <bean id="propertyConfigurer"
        class="org.springframework.beans.factory.config.PropertyPlaceholderConfigurer">
        <property name="locations">
            <list>
                <value>jdbc.properties</value>
            </list>
        </property>
    </bean>

    <bean id="dataStuff" class="DataStuff" destroy-method="bye">
        <property name="dataSource"><ref local="dataSource"/></property>
    </bean>

    <bean id="dataSource"
        class="org.apache.commons.dbcp.BasicDataSource" destroy-method="close">
        <property name="driverClassName"><value>${driverClassName}</value></property>
        <property name="url"><value>${url}</value></property>
        <property name="username"><value>${username}</value></property>
        <property name="password"><value>${password}</value></property>
    </bean>
</beans>
```

JdbcTemplate Example

What we changed:

- Extend JdbcDaoSupport for convenience
- JdbcTemplate.query() method returning a List of mapped objects.
- Added JdbcTemplate.update()
- Switched to ApplicationContext and a jdbc.properties file.

Before: [Name=Maria id=2, Name=John id=3]

After: [Name=Maria id=2, Name=Eric id=3]

JDBC Operation Example

```
import java.sql.Types;
import java.util.List;
import org.springframework.jdbc.core.SqlParameter;
import org.springframework.jdbc.core.support.JdbcDaoSupport;
import org.springframework.jdbc.object.SqlUpdate;

public class DataStuff extends JdbcDaoSupport {
    private PersonQuery personQuery;
    private SqlUpdate nameUpdate;
    private static final String updateSql = "update mytest set name = ? where id = ?";

    public List getNames() {
        return personQuery.execute(2);
    }

    public void updateAName(long id, String name) {
        nameUpdate.update(new Object[] {name, new Long(id)});
    }

    public void initDao() {
        this.personQuery = new PersonQuery(getDataSource());
        this.nameUpdate = new SqlUpdate(getDataSource(), updateSql);
        nameUpdate.declareParameter(new SqlParameter("newName", Types.VARCHAR));
        nameUpdate.declareParameter(new SqlParameter("id", Types.INTEGER));
        nameUpdate.compile();
    }
}
```

JDBC Operation Example

```
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Types;

import javax.sql.DataSource;

import org.springframework.jdbc.core.SqlParameter;
import org.springframework.jdbc.object.MappingSqlQuery;

public class PersonQuery extends MappingSqlQuery {
    private final static String querySql = "select name, id from mytest where id >= ?";

    PersonQuery(DataSource dataSource) {
        super(dataSource, querySql);
        declareParameter(new SqlParameter("id", Types.INTEGER));
    }

    protected Object mapRow(ResultSet rs, int currentRowNum) throws SQLException {
        Person person = new Person();
        person.setId(rs.getLong("id"));
        person.setName(rs.getString("name"));
        return person;
    }
}
```


JDBC Operation Example

- Initialize objects in initDao
- MappingSqlQuery
 - extend base class
 - implement mapRow()
- SqlUpdate
 - use update()

StoredProcedure Example

postgres:

```
CREATE OR REPLACE FUNCTION get_name(integer) RETURNS varchar
AS 'select 'Hello ' || name from mytest where id = $1;'
LANGUAGE SQL
IMMUTABLE
RETURNS NULL ON NULL INPUT
```

oracle:

```
CREATE OR REPLACE FUNCTION get_name(in_id integer)
RETURN varchar
AS
    v_return_value varchar(4000);
BEGIN
    select 'Hello ' || name into v_return_value from mytest where id = in_id;
    RETURN v_return_value;
END;
```

Stored Procedure Example

```
import java.sql.Types;
import java.util.HashMap;
import java.util.Map;

import javax.sql.DataSource;

import org.springframework.jdbc.core.SqlOutParameter;
import org.springframework.jdbc.core.SqlParameter;
import org.springframework.jdbc.object.StoredProcedure;

public class GetNameProc extends StoredProcedure {
    private static final String getNameSql = "get_name";

    GetNameProc(DataSource dataSource) {
        super(dataSource, getNameSql);
        setFunction(true);
        declareParameter(new SqlOutParameter("message", Types.VARCHAR));
        declareParameter(new SqlParameter("id", Types.INTEGER));
        compile();
    }

    String execute(long id) {
        Map inParams = new HashMap(1);
        inParams.put("id", new Long(id));
        Map results = execute(inParams);
        return (String)results.get("message");
    }
}
```

Stored Procedure Example

```
import org.springframework.jdbc.core.support.JdbcDaoSupport;

public class DataStuff extends JdbcDaoSupport {
    private GetNameProc getNameProc;

    public String getAName() {
        return getNameProc.execute(3);
    }

    public void initDao() {
        System.out.println("Hi!");
        this.getNameProc = new GetNameProc(getDataSource());
    }
}
```

Stored Procedure

- Extend StoredProcedure and declare in and out parameters
- create an execute method that corresponds to your stored procedure signature - put parameters in a Map
- extract the returned values

Batch Processing

```
import java.sql.Types;
import java.util.Iterator;
import java.util.List;
import org.springframework.jdbc.core.SqlParameter;
import org.springframework.jdbc.core.support.JdbcDaoSupport;
import org.springframework.jdbc.object.BatchSqlUpdate;
import org.springframework.jdbc.object.SqlUpdate;

public class DataStuff extends JdbcDaoSupport {
    private PersonQuery personQuery;
    private static final String insertSql = "insert into mytest (id, name) values(?, ?)";

    public List getNames() {
        return personQuery.execute(0);
    }

    public void insertNames(List names) {
        BatchSqlUpdate nameInsert = new BatchSqlUpdate(getDataSource(), insertSql);
        nameInsert.declareParameter(new SqlParameter("id", Types.INTEGER));
        nameInsert.declareParameter(new SqlParameter("newName", Types.VARCHAR));
        nameInsert.compile();
        Iterator nameIter = names.iterator();
        while (nameIter.hasNext()) {
            nameInsert.update((Object[]) nameIter.next());
        }
        nameInsert.flush();
    }

    public void initDao() {
        this.personQuery = new PersonQuery(getDataSource());
    }
}
```

Batch Processing

```
import java.util.ArrayList;
import java.util.List;

import org.springframework.context.support.ClassPathXmlApplicationContext;

public class JustStuff {

    public static void main(String[] args) {
        ClassPathXmlApplicationContext ctx;

        ctx = new ClassPathXmlApplicationContext("dataAccessContext.xml");
        DataStuff test = (DataStuff)ctx.getBean("dataStuff");

        System.out.println("Before: " + test.getNames());

        List names = new ArrayList();
        names.add(new Object[] {new Long(4), "Foo"});
        names.add(new Object[] {new Long(5), "Bar"});
        test.insertNames(names);

        System.out.println("After: " + test.getNames());

        ctx.close();
        System.out.println("OK!");
    }
}
```


Batch Processing

- Create a new instance of BatchSqlUpdate and declare the parameters
 - ▶ **NOTE: this class is NOT thread safe.**
- Pass in a List of arrays containing the parameters
- Call flush() when the batch is done

The Bigger Picture

Spring & Persistence

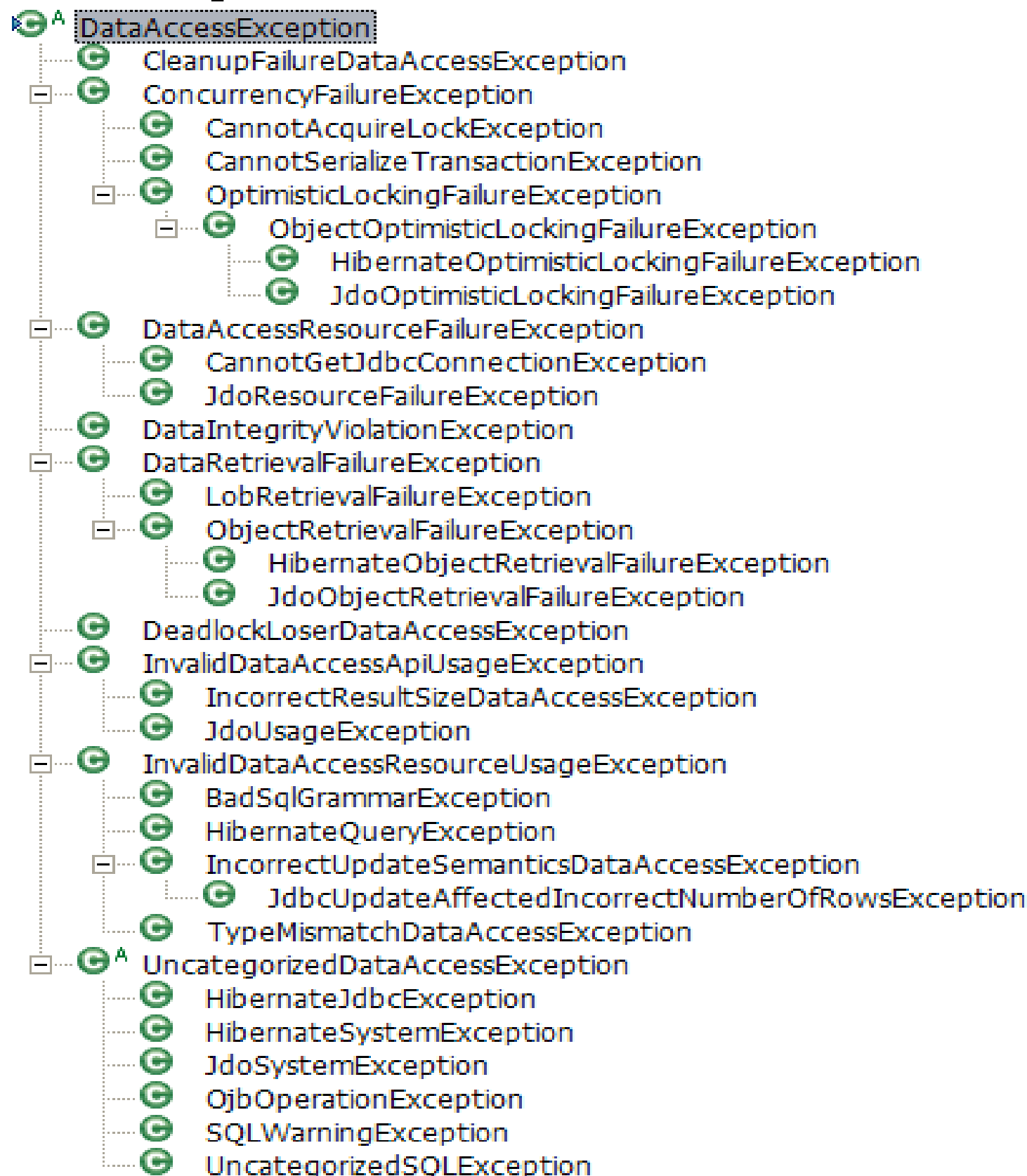
- Provide a common persistence approach across a wide range of technologies
 - JDBC
 - JDO
 - Hibernate
 - TopLink etc.
- Make persistence APIs much easier to use
 - Eliminate common causes of bugs in application code

The Bigger Picture

Spring & Persistence

- Spring encourages the use of a DAO layer which provides:
 - Common exception hierarchy based on `DataAccessException`
 - Provided via customizable exception translation
 - Common transaction management
 - Allows interoperability between different data access technologies

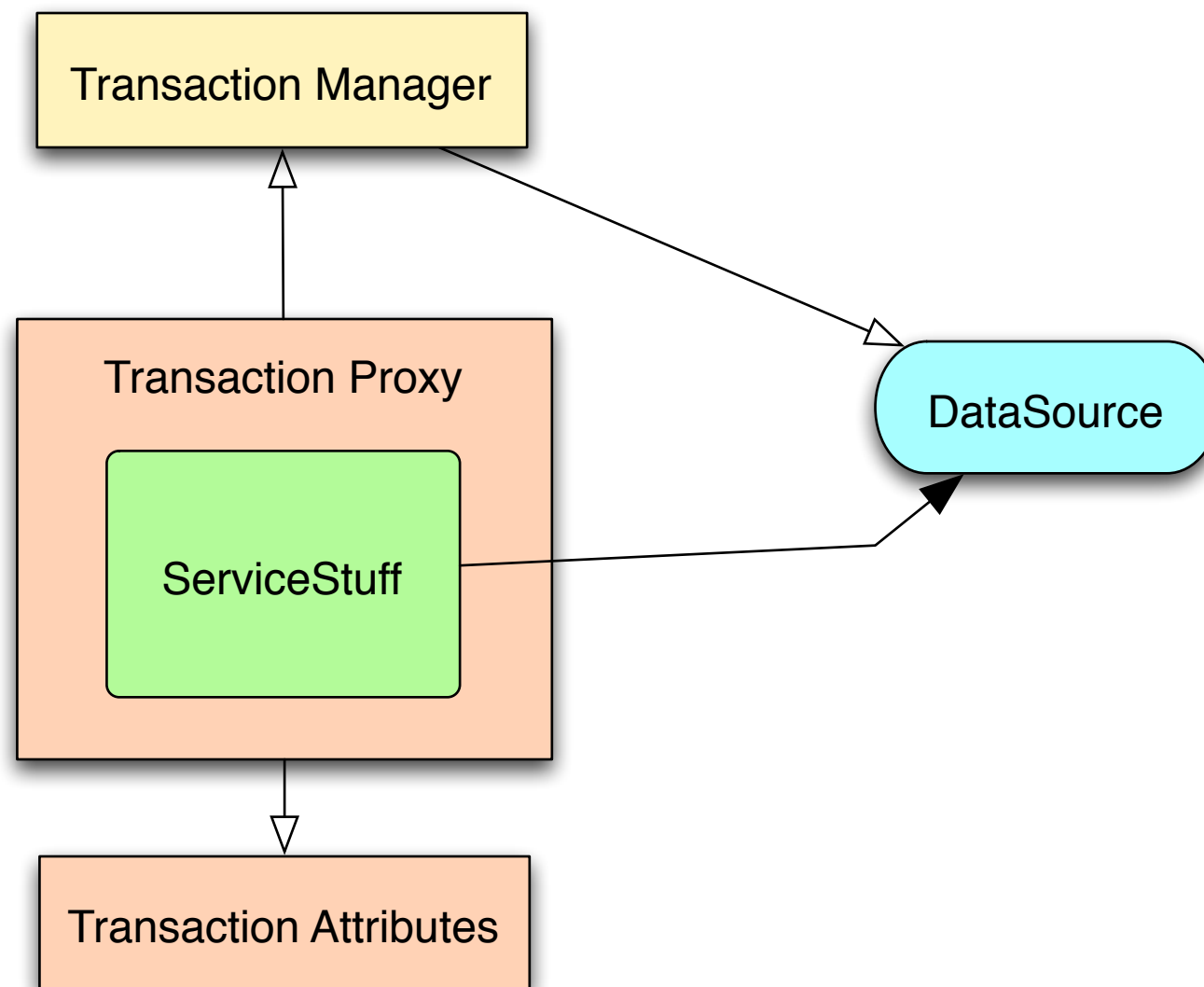
Exception Translation



Exception Translation

```
<bean id="MyDatabase" class="org.springframework.jdbc.support.SQLErrorCodes">
  <property name="badSqlGrammarCodes">
    <value>11,24,33</value>
  </property>
  <property name="dataIntegrityViolationCodes">
    <value>1,12,17,22</value>
  </property>
  <property name="customTranslations">
    <list>
      <bean
        class="org.springframework.jdbc.support.CustomSQLErrorCodesTranslation">
        <property name="errorCodes">
          <value>942</value></property>
        <property name="exceptionClass">
          <value>com.mycompany.test.MyCustomException</value>
        </property>
      </bean>
    </list>
  </property>
</bean>
```

Transaction Management



Transaction Management

- Use interfaces for you business service layer
- Add transaction proxy and transaction attributes for business/service methods
- Pick a transaction manager for your environment --
DataSourceTransactionManger /
HibernateTransactionManager /
JtaTransactionManager

Transaction Management

```
<bean id="dataSource"
...
</bean>

<bean id="transactionManager"
  class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
  <property name="dataSource"><ref local="dataSource"/></property>
</bean>

<bean id="dataStuff" class="DataStuffImpl">
  <property name="dataSource"><ref local="dataSource"/></property>
</bean>

<bean id="serviceStuff"
  class="org.springframework.transaction.interceptor.TransactionProxyFactoryBean">
  <property name="transactionManager"><ref local="transactionManager"/></property>
  <property name="target">
    <bean class="ServiceStuffImpl">
      <property name="dataStuff"><ref local="dataStuff"/></property>
    </bean>
  </property>
  <property name="transactionAttributes">
    <props>
      <prop key="*">PROPAGATION_REQUIRED</prop>
    </props>
  </property>
</bean>
```


Accessing the ApplicationContext

- In a Standalone Environment

```
WebApplicationContext ctx =  
    new ClassPathXmlApplicationContext("applicationContext.xml");
```

- In a Web Environment with access to ServletContext

```
ServletContext sc =  
    request.getSession().getServletContext();  
WebApplicationContext ctx =  
    WebApplicationContextUtils.getWebApplicationContext(sc);
```


Accessing the ApplicationContext

- In a Web Environment without access to ServletContext - Definition

beanRefFactory.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN//EN"
    "http://www.springframework.org/dtd/spring-beans.dtd">
<beans>
  <bean id="mycontext"
    class="org.springframework.context.support.ClassPathXmlApplicationContext">
    <constructor-arg>
      <list>
        <value>com/mycompany/myapp/util/applicationContext.xml</value>
      </list>
    </constructor-arg>
  </bean>
</beans>
```

Accessing the ApplicationContext

- In a Web Environment without access to ServletContext - Access

...

```
BeanFactoryLocator bfl =  
    SingletonBeanFactoryLocator.getInstance();  
BeanFactoryReference bf =  
    bfl.useBeanFactory("mycontext");  
ApplicationContext ctx = bf.getFactory();
```

...

Testing Framework

```
import java.util.ArrayList;
import java.util.List;

import org.springframework.test.AbstractTransactionalDataSourceSpringContextTests;

public class DataStuffTests extends AbstractTransactionalDataSourceSpringContextTests {
    protected DataStuff dataStuff;

    // used to populate test class instance using dependency injection
    public void setDataStuff(DataStuff dataStuff) {
        this.dataStuff = dataStuff;
    }

    protected String[] getConfigLocations() {
        return new String[] {"dataAccessContext.xml"};
    }

    public void testAddNames() {
        List l1 = dataStuff.getNames();
        int x = l1.size();
        List names = new ArrayList();
        names.add(new Object[] {new Long(4), "Foo"});
        names.add(new Object[] {new Long(5), "Bar"});
        dataStuff.insertNames(names);
        List l2 = dataStuff.getNames();
        int y = l2.size();
        assertEquals("Two rows were added", 2, y-x);
    }
}
```

Testing Framework

...

```
<bean id="dataSource"
```

```
    ...  
</bean>
```

```
<!-- Transaction manager for a single JDBC DataSource (alternative to JTA) -->
```

```
<bean id="transactionManager"  
    class="org.springframework.jdbc.datasource.DataSourceTransactionManager">  
    <property name="dataSource"><ref local="dataSource"/></property>  
</bean>
```

```
<bean id="dataStuff"  
    class="org.springframework.transaction.interceptor.TransactionProxyFactoryBean">  
    <property name="transactionManager"><ref local="transactionManager"/></property>  
    <property name="target">  
        <bean class="DataStuffImpl">  
            <property name="dataSource"><ref local="dataSource"/></property>  
        </bean>  
    </property>  
    <property name="transactionAttributes">  
        <props>  
            <prop key="*">PROPAGATION_REQUIRED</prop>  
        </props>  
    </property>  
</bean>
```

...

Testing Framework

- Extend one of the `AbstractSpringContextTests` implementations
- implement `getConfigLocations` and expose beans you want to have injected
- write your tests
- wrap class in a transaction proxy if you want transaction support including automatic rollback

New JDBC Features

- **Spring 1.2 will introduce some new JDBC features:**

- ✓ Named SQL Parameters

```
private final static String querySql =  
    "select name, id from mytest where id >= :id";
```

- ✓ RowSet support in queryForRowSet

```
jdbcTemplate.queryForRowSet("select * from mytest")
```

Other New Features

- **Spring 1.2 Features:**
 - ✓ Hibernate 3 Support
 - ✓ TopLink Support and (possibly) Cayenne Support
 - ✓ JMX (not an MBean server, just stuff to simplify writing MBeans)
 - ✓ Transaction annotations with Java 5
 - ✓ Spring Web Flow

Spring Community

- www.springframework.org
- 17 developers
- Around 8 “core” developers
- Significant number of contributors
- Framework is developed test-first
- Vibrant community
- Very active mailing lists and forums
- JIRA Issue tracker at Atlassian
- Approaching 150,000 downloads with Spring 1.1.4 release

Spring Books

- **Many books on Spring in 2004/2005**
 - ✦ J2EE Without EJB (Johnson/Hoeller)
 - ✦ Better, Faster, Lighter Java (Tate/Gehtland)
 - ✦ Spring Live: Matt Raible
 - ✦ Pro Spring from APress (Harrop/Machacek)
 - ✦ Manning Spring in Action (Walls/Breidenbach)
 - ✦ Professional Spring Development (Q2 2005):
Johnson/Risberg/Hoeller/Arendsen
 - ✦ Spring Developer's Handbook (Bruce Tate,
O'Reilly, 2005)
 - ✦ O'Reilly, APress and Addison-Wesley publishing
additional books

Published-----

Support

- Support Forum www.springframework.org
- Reference Document www.springframework.org/docs/reference.pdf
- Professional support provided by Interface21 headed by Rod Johnson & Juergen Hoeller with 8 consultants worldwide www.springframework.com

Related Projects

- **Spring Rich Client Project**
 - ✓ Brings Spring IoC container to Swing applications
- **Acegi Security for Spring**
 - ✓ Powerful security solution for Spring applications
 - ✓ Much more flexible than out of the box J2EE security
- **Spring.NET**
 - ✓ Ports core IoC and AOP concepts to .NET, offering a consistent programming model for Java and .NET projects